Laboratorio 9: Respuestas

Deep Learning y Sistemas Inteligentes - CC3092

José Antonio Mérida Castejón

11 de octubre de 2026

> **Nota:** este informe fue generado con IA (Claude Code) a partir del notebook `Notebook.ipynb`, transcribiendo sus respuestas, tablas y figuras. El detalle del uso de IA está en `AI.md`.

El código, las salidas y las gráficas de todos los tasks ya están en el notebook `Notebook.ipynb`; este documento contiene las respuestas a cada task. Semilla fija: `SEED = 42` (Python, NumPy y PyTorch/CUDA).

Task 1 — Autoencoder variacional

Task 1.1 — Datos

Fashion-MNIST escalado a $[0, 1]$, últimos 5 000 ejemplos de train como validación (train: 55 000, val: 5 000). Configuración: `batch_size = 128`, `latent_dim = 2`, `epochs = 20`, `lr = 1e-3`, `beta_values = [0.1, 1.0, 10.0]`.

Task 1.2 — Modelo y pérdida

Funciones implementadas:

- `reparameterize`: $z = \mu + \sigma \odot \epsilon$
- `kl_divergence`: KL cerrada entre $\mathcal{N}(\mu, \text{diag}(\sigma^2))$ y $\mathcal{N}(0, I)$, sumada sobre dimensiones latentes → un valor **por imagen**
- `vae_loss`: $\|x - \hat{x}\|^2$ (suma sobre 784 píxeles) + βD_{KL}

Arquitectura: VAE completamente conectado, `latent_dim = 2`.

- **Encoder:** Flatten → 784→512 → ReLU → 512→256 → ReLU
- **Heads:** `fc_mu` y `fc_lv`, cada uno 256→2 (μ y $\log \sigma^2$)
- **Decoder:** 2→256 → ReLU → 256→512 → ReLU → 512→784 → Sigmoid → (1, 28, 28)

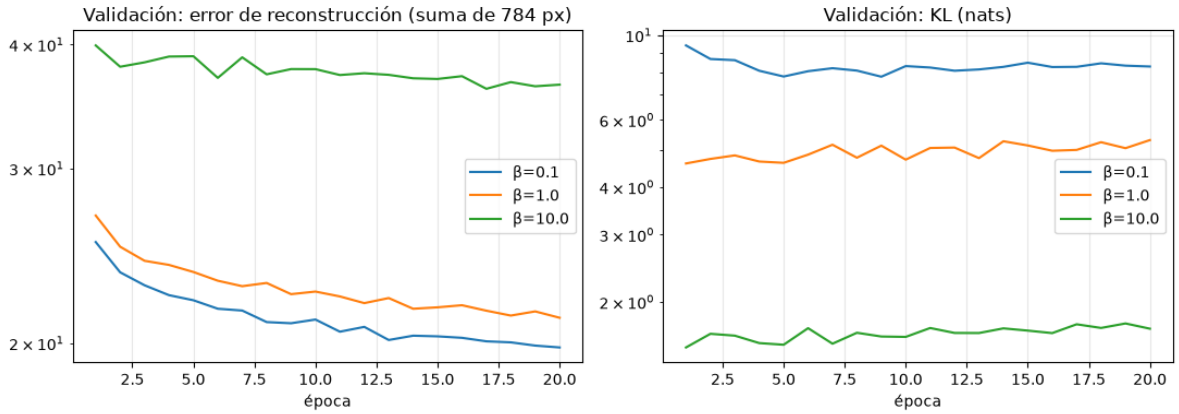
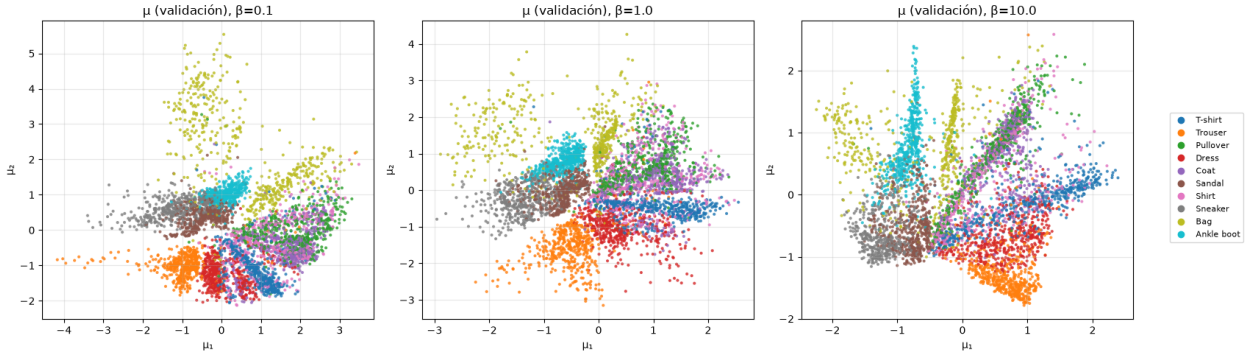


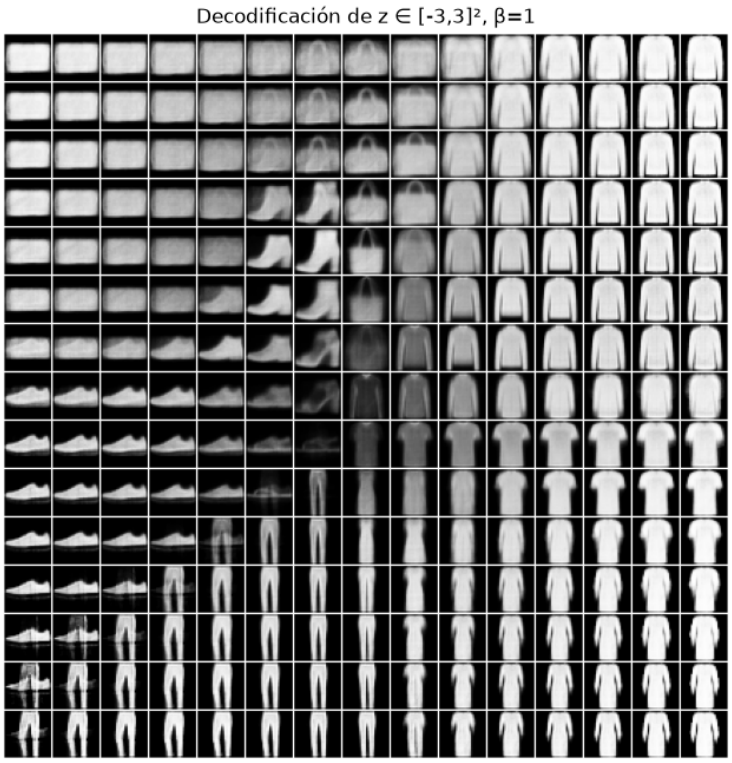
Figura 1: Curvas de validación del error de reconstrucción y de la KL por época, para cada β.

Task 1.3 — Figuras

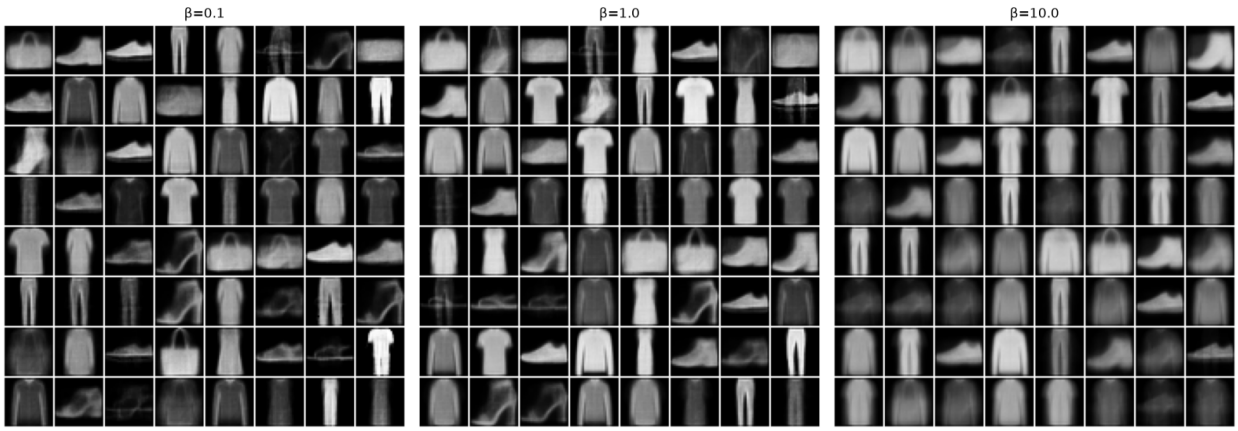
(a) Dispersión de μ en validación, coloreada por clase.



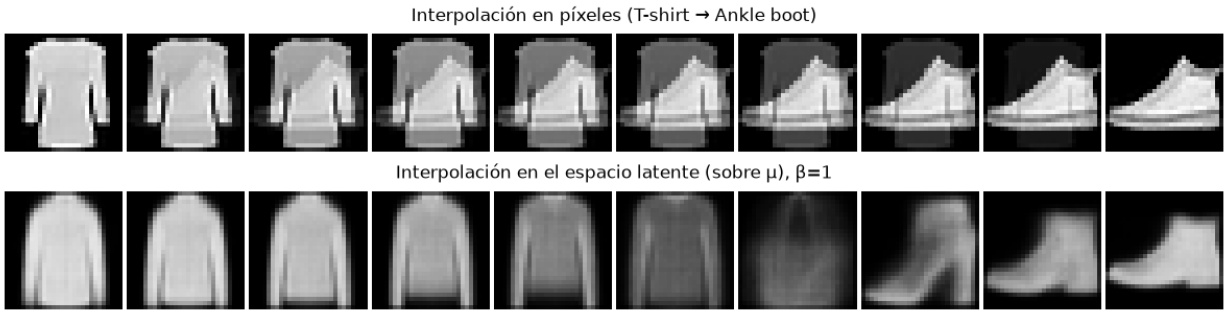
(b) Cuadrícula 15×15 de $z \in [-3, 3]^2$ decodificada, $\beta = 1$ (la fila superior corresponde a $z_2 = +3$).



(c) 64 imágenes generadas con $z \sim \mathcal{N}(0, I)$ por cada β.



(d) Interpolación entre dos imágenes de validación de clases distintas (8 puntos intermedios), $\beta = 1$: píxeles vs. espacio latente (sobre μ).



Task 1.4 — Análisis

(a) Reconstrucción y KL finales de validación, KL en bits vs. $\log_2 10 \approx 3.32$ **bits**

β	recon (val)	KL nats (val)	KL bits (val)
0.1	19.808	8.277	11.941
1.0	21.247	5.308	7.658
10.0	36.765	1.700	2.452

Al subir β la reconstrucción empeora y el KL baja: recon 19.8 $\rightarrow$ 21.2 $\rightarrow$ 36.8 y KL 8.28 $\rightarrow$ 5.31 $\rightarrow$ 1.70 nats (11.9 $\rightarrow$ 7.7 $\rightarrow$ 2.5 bits) para $\beta = 0.1$, 1, 10. Con β pequeño el modelo prioriza reconstruir y se aleja del prior; con β grande el término KL domina y el código latente transmite menos información.

Para $\beta = 0.1$ y $\beta = 1$ el KL (11.9 y 7.7 bits) supera $\log_2 10 \approx 3.32$ bits, así que hay capacidad de sobra para codificar la clase. Para $\beta = 10$ el KL es 2.45 bits < 3.32, el código no alcanza para identificar la clase de una imagen de forma fiable y además esos bits deben describir también la forma y el estilo. Esto coincide con el diagrama de dispersión de $\beta = 10$: las clases de forma muy distinta (Trouser, Bag, Ankle boot, Sneaker/Sandal) ocupan regiones propias, pero T-shirt, Pullover, Coat, Shirt y Dress se superponen en una sola zona. Es decir, el código distingue solo $\sim$5–6 grupos ($2^{2.45} \approx 5.5$) en lugar de 10 clases.

(b) KL por Monte Carlo vs. KL cerrada

KL por Monte Carlo (10^5 muestras de $q(z|x)$, promedio de $\log q(z) - \log p(z)$) vs. KL cerrada, $\beta = 1$, una imagen de validación.

Para una imagen de validación, con el modelo $\beta = 1$ ($\mu = [1.761, 0.214]$, $\sigma = [0.164, 0.079]$):

Método	KL (nats)
Forma cerrada	4.93679
Monte Carlo (10^5 muestras)	4.93691 ± 0.00324

Error relativo: 2.4×10^{-5} (diferencia absoluta $\approx 1.2 \times 10^{-4}$).

¿Es compatible con la variabilidad Monte Carlo? Sí. El error estándar del estimador es $\text{SE} = s/\sqrt{N} \approx 0.0032$, y la diferencia observada es solo 0.04 SE. Una fluctuación típica sería de

alrededor de 1 SE, así que la diferencia está muy dentro de lo esperado.

Esto confirma que la fórmula cerrada

$$D_{KL} = \tfrac{1}{2} \sum_j \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2\right)$$

está bien implementada.

(c) Región del espacio latente no reconocible; punto medio de las interpolaciones

Con $\beta = 1$ las imágenes de las esquinas de la cuadrícula siguen siendo reconocibles. Bolsos arriba a la izquierda, zapatos en el centro-izquierda, pantalones abajo a la izquierda y prendas superiores a la derecha. La región no reconocible está cerca del origen, en la frontera entre las zonas de zapato (Sneaker/Ankle boot), pantalón y prendas superiores (aprox. $z_1 \in [-1, 0]$, $z_2 \in [-1, 0.5]$). Ahí el decoder produce imágenes oscuras y tenues, sin forma de ninguna clase.

El término KL empuja los posteriores $q(z|x)$ hacia $\mathcal{N}(0, I)$, de modo que las clases quedan juntas y solapadas alrededor del origen. En una frontera, el mismo z puede venir de imágenes de clases distintas y como la pérdida es un error cuadrático, el decoder aprende a producir el promedio de ellas. Con β más grande el overlap sería mayor y más zonas se verían así; con β pequeño habría huecos con menos datos donde el decoder se entrena poco.

En píxeles, el punto medio es una superposición lineal de las dos imágenes, se ven a la vez las mangas de la camiseta y la silueta de la bota, cada una al 50 % de intensidad. Esto claramente no es una imagen plausible de ninguna clase. En el espacio latente, el punto medio $\mu = \frac{1}{2}(\mu_a + \mu_b)$ se decodifica con una red no lineal, así que no hay superposición. La secuencia se mantiene como una prenda de manga larga que se va desvaneciendo, pasa por una mancha oscura y borrosa (el punto medio cae en la frontera entre clases, la región descrita arriba) y luego cambia de golpe a una bota. Difieren porque la interpolación en píxeles es lineal en el espacio de imagen, mientras que la latente es lineal en z pero pasa por el decoder, que mapea la zona entre clases a su "imagen promedio" borrosa y no a una mezcla de ambas.

(d) Nitidez y relación con σ_x^2

Nitidez (media de $|x_{j+1} - x_j|$, 1 000 imágenes cada una):

Real	VAE $\beta = 0.1$	VAE $\beta = 1$	VAE $\beta = 10$
0.0891	0.0446	0.0501	0.0439

Las muestras del VAE tienen alrededor de la mitad de la nitidez de las imágenes reales.

Relación con σ_x^2. Con decoder gaussiano, el negativo del ELBO es proporcional a $\|x - \hat{x}\|^2 + 2\sigma_x^2 D_{KL}$. Mi pérdida es $\mathcal{L}_\beta = \|x - \hat{x}\|^2 + \beta D_{KL}$, así que $2\sigma_x^2 = \beta$, es decir, $\sigma_x^2 = \beta/2$.

Por qué salen borrosas. Con $\beta = 1$, el modelo supone un ruido por píxel de $\sigma_x \approx 0.7$, enorme para imágenes en $[0, 1]$. Por eso casi no se penaliza producir imágenes suavizadas. Además, al entrenar con error cuadrático, el decoder aprende la media $\mathbb{E}[x \mid z]$. Esa media promedia los bordes y texturas posibles, y el resultado es borroso. Esto explica que todas las muestras pierdan cerca de la mitad de la nitidez.

La diferencia entre valores de β es pequeña y no monótona (0.044–0.050, una sola semilla), así que no permite sacar conclusiones sobre el efecto de β.

Task 2 — Proceso forward de difusión

Task 2.1 — Calendario y fórmula cerrada

Datos escalados a $[-1, 1]$, con $T = 1000$, $\beta_{\text{start}} = 10^{-4}$ y $\beta_{\text{end}} = 0.02$. Convención de índices: los tensores del calendario tienen longitud $T + 1$ y se indexan **directamente por** $t \in \{1, \ldots, T\}$; la posición 0 es un relleno ($\beta_0 = 0,\ \alpha_0 = \bar{\alpha}_0 = 1$, es decir x_0). La fórmula cerrada implementada es $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$. Para el calendario lineal, $\bar{\alpha}_T = 4.0358 \times 10^{-5}$.

Task 2.2 — Verificación empírica de la fórmula cerrada

5 000 copias de una imagen x_0, 300 pasos $x_s = \sqrt{\alpha_s} x_{s-1} + \sqrt{\beta_s}\epsilon_s$ con ruido independiente, y comparación contra $\sqrt{\bar{\alpha}_{300}} x_0$ y $1 - \bar{\alpha}_{300}$.

Cantidad	Valor
$\bar{\alpha}_{300}$	0.39642
Varianza teórica $1 - \bar{\alpha}_{300}$	0.60358
Error máximo $\lvert\text{media}_{\text{emp}} - \text{media}_{\text{teo}}\rvert$	0.03352
Error estándar esperado $\sigma/\sqrt{N}$	0.01099
Varianza empírica media	0.60376
$\lvert\text{var}_{\text{emp}} - \text{var}_{\text{teo}}\rvert$ medio	0.00936

Sí, es compatible. Cada píxel de la media empírica promedia $N = 5000$ muestras con varianza $1 - \bar{\alpha}_{300} = 0.60358$, así que

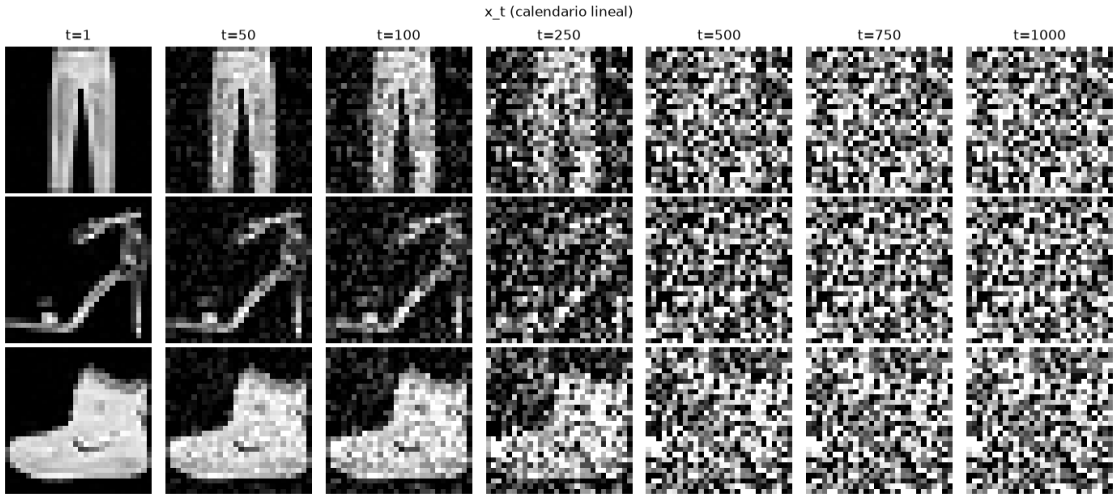
$$\text{SE} = \sqrt{0.60358/5000} = 0.011.$$

El error máximo observado es $0.0335 \approx 3\,\text{SE}$. Como es el máximo sobre 784 píxeles, eso es lo esperado: el máximo de 784 normales suele estar cerca de $3.2\,\text{SE}$.

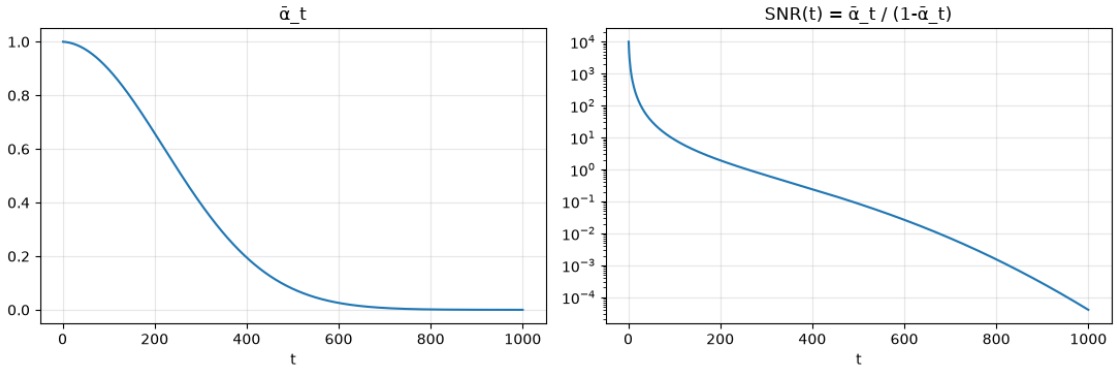
La varianza también coincide: 0.60376 empírica frente a 0.60358 teórica, con un error medio por píxel de 0.0094. El error estándar de una varianza muestral es $(1 - \bar{\alpha}_{300})\sqrt{2/(N-1)} \approx 0.012$, así que el error medio es ≈ 0.8 veces ese SE, justo lo esperado ($\mathbb{E}|Z| \approx 0.8$ para $Z \sim \mathcal{N}(0, 1)$).

Task 2.3 — Visualización y SNR

(a) x_t para $t \in \{1, 50, 100, 250, 500, 750, 1000\}$ (Trouser, Sandal, Ankle boot).



(b) $\bar{\alpha}_t$ y $\mathrm{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$ (escala logarítmica).



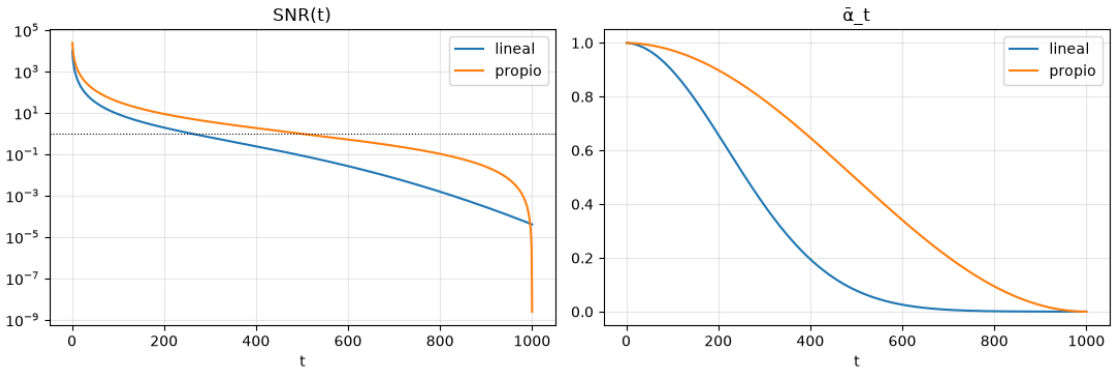
(c)

- Primer t con $\mathrm{SNR}(t) < 1$: $\mathbf{t = 260}$
- Primer t con $\bar{\alpha}_t < 0.01$: $\mathbf{t = 674}$

Task 2.4 — Segundo calendario

Claude fue el encargado de implementar el segundo calendario, se le indicó que tomara alguno popular, lo citara e implementara con alguna modificación.

Calendario coseno de Nichol & Dhariwal (2021) [1], $\bar{\alpha}_t = f(t)/f(0)$, $f(t) = \cos^2\left(\frac{t/T+s}{1+s} \cdot \frac{\pi}{2}\right)$, $s = 0.008$, con $\beta_t = 1 - \bar{\alpha}_t/\bar{\alpha}_{t-1}$ recortado a $[10^{-5}, 0.999]$. Cumple $\bar{\alpha}_T < 10^{-3}$ y $\beta_t \in (0, 1)$.



Calendario	$\bar{\alpha}_T$	Primer t con SNR < 1
Lineal	4.0358×10^{-5}	260
Propio (coseno)	2.4288×10^{-9}	497

Task 2.5 — Análisis

(a) Aproximación $\log \bar{\alpha}_T \approx -\sum_t \beta_t - \frac{1}{2}\sum_t \beta_t^2$

Con β_t lineal de 10^{-4} a 0.02 y $T = 1000$:

$$\sum_t \beta_t = \frac{T(\beta_1 + \beta_T)}{2} = 10.05, \qquad \sum_t \beta_t^2 \approx \frac{T}{3}\left(\beta_1^2 + \beta_1\beta_T + \beta_T^2\right) = 0.1340.$$

$$\log \bar{\alpha}_T \approx -10.05 - \tfrac{1}{2}(0.1340) = -10.117 \;\Rightarrow\; \bar{\alpha}_T \approx 4.04 \times 10^{-5}.$$

El valor exacto del código es 4.0358×10^{-5} (diferencia de $\approx 0.07\,\%$, debida al término de tercer orden omitido). Las sumas calculadas en la celda coinciden con las hechas a mano.

(b) Pasos con SNR $\in [0.01, 100]$ por calendario

Calendario	SNR $\in [0.01, 100]$	SNR < 0.01	SNR > 100
Lineal	647	326	27
Coseno	880	64	56

- **SNR < 0.01:** $x_t \approx \epsilon$, así que predecir ϵ es trivial y aporta poco gradiente. Con t uniforme, en el lineal son el 33 % de los sorteos, casi desperdiciados.
- **SNR > 100:** $x_t \approx x_0$, así que aislar el ruido es muy difícil (es el intervalo con mayor MSE en la Task 3.2). Son pocos pasos (27 en el lineal, 2.7 % de los sorteos), así que su aporte total al entrenamiento es pequeño.
- **SNR $\in [0.01, 100]$:** señal y ruido son comparables, y la tarea es informativa.

El coseno es más eficiente: el 88 % de sus pasos están en el rango útil, frente al 65 % del lineal, y desperdicia muchos menos en ruido puro (64 frente a 326).

(c) $\mathrm{Var}(x_t) = \bar{\alpha}_t \mathrm{Var}(x_0) + (1 - \bar{\alpha}_t)$

Como $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$, con ϵ independiente de x_0 (covarianza nula) y $\mathrm{Var}(\epsilon) = 1$:

$$\mathrm{Var}(x_t) = \bar{\alpha}_t\, \mathrm{Var}(x_0) + (1 - \bar{\alpha}_t)\, \mathrm{Var}(\epsilon) + 2\sqrt{\bar{\alpha}_t(1 - \bar{\alpha}_t)}\, \mathrm{Cov}(x_0, \epsilon) = \bar{\alpha}_t\, \mathrm{Var}(x_0) + (1 - \bar{\alpha}_t).$$

Verificación con $\mathrm{Var}(x_0) = 0.49826$:

t	Empírica	Predicción
1	0.4983	0.4983
250	0.7369	0.7370
500	0.9604	0.9606
1000	1.0003	1.0000

Como $\bar{\alpha}_T \approx 0$, se tiene $\mathrm{Var}(x_T) \approx 1$ sin importar los datos. Esto es necesario porque el muestreo empieza en $x_T \sim \mathcal{N}(0, I)$. Si el proceso forward terminara en otra distribución, habría un desajuste entre entrenamiento y generación.

Task 3 — Difusión condicional y guía sin clasificador

Task 3.1 — U-Net condicional

Entrada x_t `(B,1,28,28)`, instante t `(B,)` (valores $1..T$) y clase c `(B,)` en $\{0, \ldots, 10\}$ ($10 =$ condición nula). Dos niveles (28×28 con 32 canales; 14×14 con 64), cuello de botella 14×14, subida con *skip connections* concatenadas, convolución final a 1 canal. Embedding sinusoidal de t (misma fórmula de la Semana 6) + MLP, `nn.Embedding(11, d)` para la clase; la suma se inyecta con *broadcasting* en cada bloque.

Esta sección fue implementada utilizando IA, detallado en `AI.md`.

La U-Net condicional tiene **364,257 parámetros** entrenables.

Task 3.2 — Entrenamiento

Por iteración: $t \sim U\{1, \ldots, T\}$, $\epsilon \sim \mathcal{N}(0, I)$, x_t con `q_sample`, clase $\to 10$ con probabilidad `p_uncond`, MSE entre ϵ y la predicción (`epochs = 30`, `batch_size = 128`, `lr = 2e-4`, `p_uncond = 0.1`).

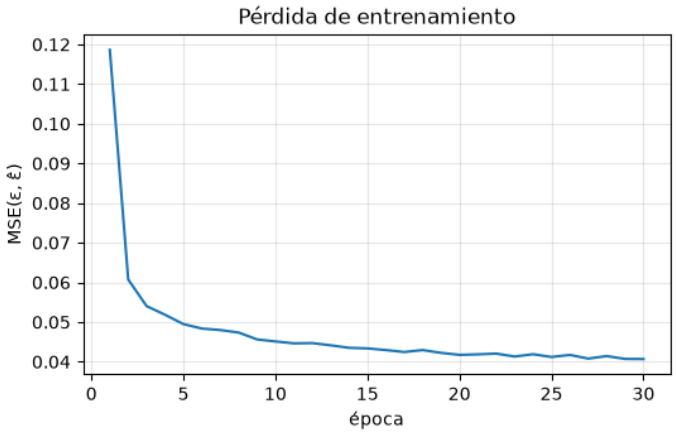


Figura 2: Pérdida de entrenamiento por época.

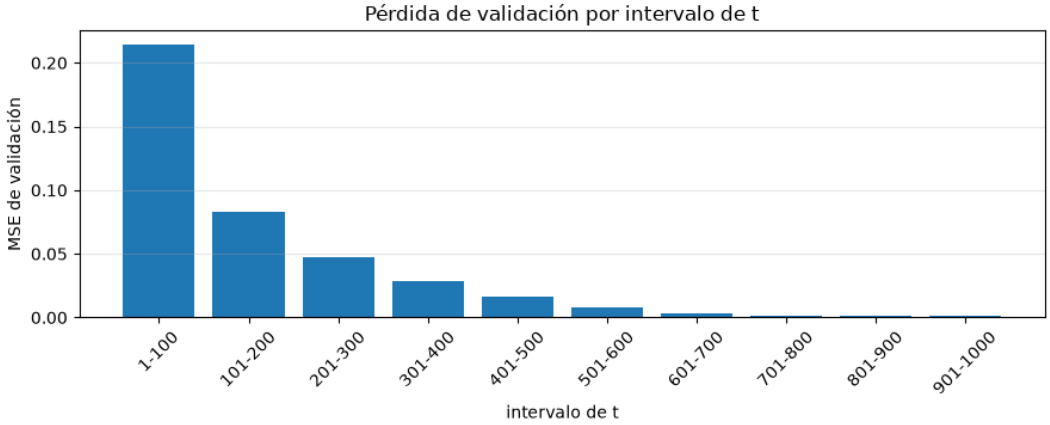


Figura 3: Pérdida de validación (5 000 imágenes) por intervalo de t de 100 pasos.

Task 3.3 — Muestreo ancestral

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\hat{\epsilon}\right) + \sigma_t z, \quad \sigma_t^2 = \beta_t,\ z = 0 \text{ en el último paso}$$

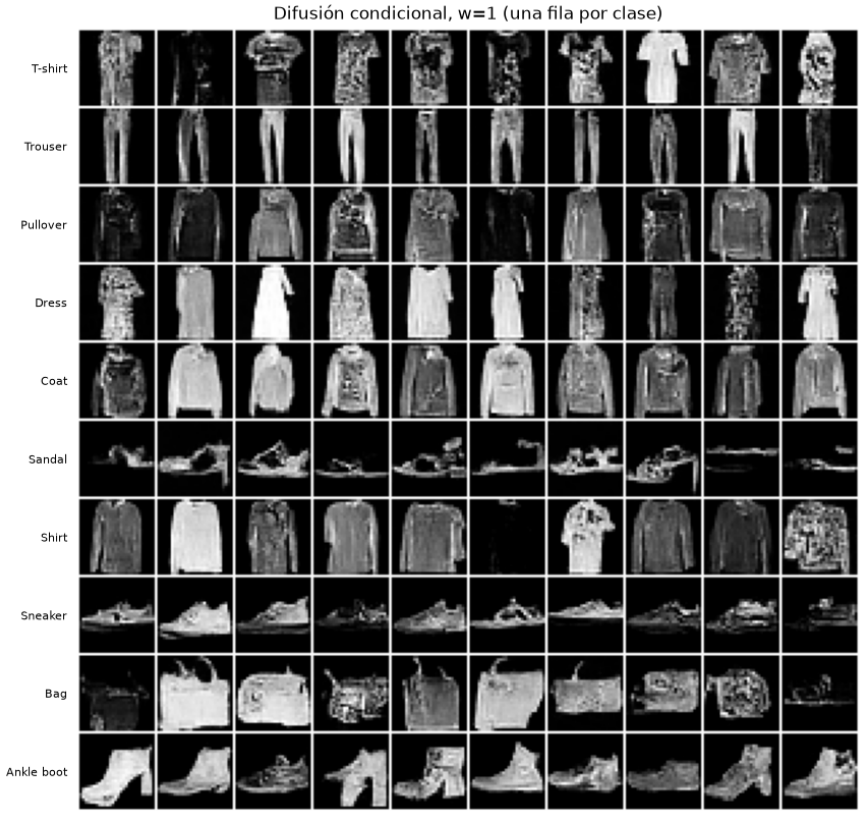


Figura 4: 10 imágenes por clase con $w = 1$, una fila por clase.

Task 3.4 — Guía sin clasificador

(a) Predicción guiada, con $w = 1$ equivalente al modelo condicional puro:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + w\,(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing))$$

(b–c) 50 imágenes por clase para cada $w \in \{1, 3, 7\}$. Clasificador sencillo entrenado sobre imágenes reales hasta $\geq 88\,\%$ de exactitud en validación (exactitud obtenida: 0.8874). Diversidad = distancia euclidiana media entre pares de imágenes generadas de la misma clase (promediada sobre clases); referencia con 50 imágenes reales por clase.

(d)

w	fidelidad	diversidad	s / 100 imgs
1	0.784	7.937	2.807
3	0.952	7.880	6.831
7	0.878	7.592	6.133

Diversidad de referencia (real, 50/clase): 8.838.

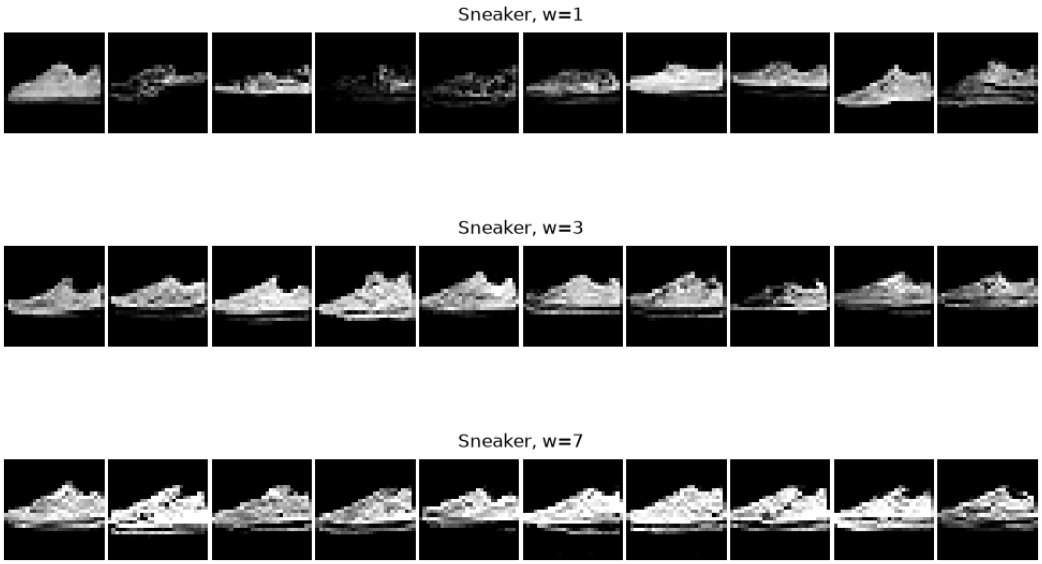


Figura 5: 10 muestras de Sneaker para cada w.

Task 3.5 — Análisis

(a) Cambio de fidelidad y diversidad con w; clase cuya diversidad más cae entre $w = 1$ y $w = 7$

Clase	real	$w = 1$	$w = 3$	$w = 7$	caída $w1 \to w7$ (%)
T-shirt	9.371	7.623	9.735	11.727	−53.8
Trouser	7.235	6.412	5.440	5.095	20.5
Pullover	9.279	7.037	7.018	6.666	5.3
Dress	8.056	8.591	7.805	8.806	−2.5
Coat	8.448	8.771	6.548	5.175	41.0
Sandal	9.458	6.909	8.641	12.377	−79.2
Shirt	9.997	9.403	9.740	6.961	26.0
Sneaker	6.372	6.066	6.021	6.707	−10.6
Bag	11.320	10.575	10.599	5.208	50.8
Ankle boot	8.839	7.988	7.251	7.197	9.9

w	fidelidad	diversidad	s / 100 imgs
1	0.784	7.937	2.8
3	0.952	7.880	6.8
7	0.878	7.592	6.1

(Diversidad de referencia con 50 imágenes reales por clase: 8.838.)

Tendencia. La fidelidad sube de 0.784 a 0.952 al pasar de $w = 1$ a $w = 3$ (+17 puntos), pero baja a 0.878 con $w = 7$: pasarse de guía deja de ayudar. La diversidad media cae de forma monótona

(7.937 → 7.880 → 7.592, −4.3 % de $w = 1$ a $w = 7$) y ya con $w = 1$ queda por debajo de la real (8.838). Además, $w \neq 1$ duplica el batch de la red en cada paso, por eso el tiempo pasa de 2.8 a 6–7 s por 100 imágenes.

Clase más afectada: Bag, cuya diversidad cae de 10.575 ($w = 1$) a 5.208 ($w = 7$), −50.8 %. Le siguen Coat (8.771 → 5.175, −41.0 %), Shirt (−26.0 %) y Trouser (−20.5 %).

Hipótesis (respaldada por los datos). Bag es la clase con mayor diversidad real (11.320, la más alta de las 10): sus imágenes varían muchísimo (formas, asas, patrones), así que tiene más diversidad que perder. La guía empuja hacia lo que el modelo considera más típico de la clase; en una clase tan heterogénea eso colapsa a unos pocos prototipos, y por eso cae más: con $w = 7$ su diversidad (5.208) queda en menos de la mitad de la real (11.320).

Matiz sobre la métrica. Algunas clases suben (T-shirt +54 %, Sandal +79 %, Sneaker +11 %). Con $w = 7$ las imágenes salen saturadas y de mayor contraste, y la distancia euclidiana en píxeles crece con la intensidad, así que la métrica puede subir aunque la variedad de formas baje. Sneaker es el caso más claro: en la figura, con $w = 7$ las 10 muestras son casi la misma zapatilla de alto contraste, pero su diversidad medida sube de 6.066 a 6.707. Es una limitación de usar distancia en píxeles como medida de diversidad.

(b) Matriz de confusión del clasificador sobre las imágenes generadas con $w = 1$

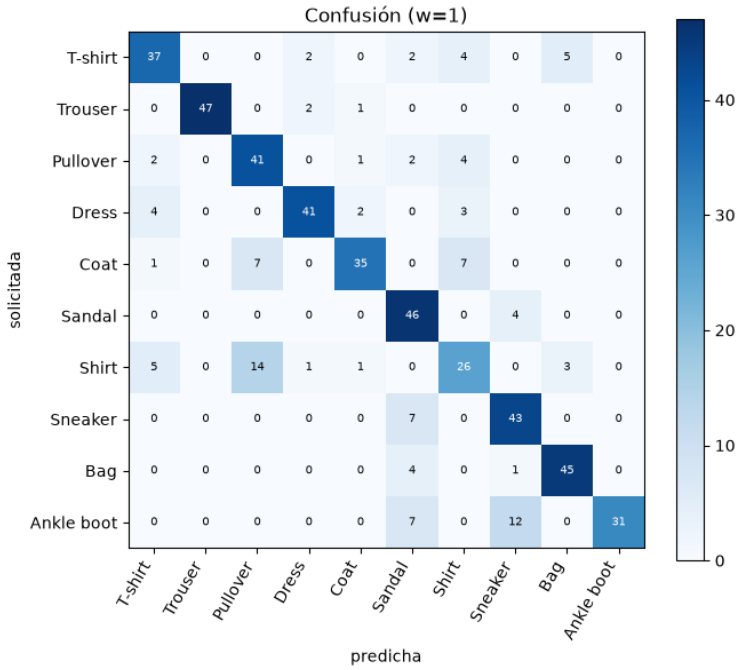




Figura 6: Muestras generadas de Shirt (arriba) y Pullover (abajo), $w = 1$.

La exactitud global con $w = 1$ es $392/500 = 0.784$ (suma de la diagonal). El par más confundido es **Shirt → Pullover** (14 de 50 imágenes pedidas como Shirt). Shirt es además la clase peor lograda: solo 26/50 (52 %) se clasifican como Shirt. Le siguen Ankle boot → Sneaker (12), y con 7 cada una: Coat → Pullover, Coat → Shirt, Sneaker → Sandal y Ankle boot → Sandal.

¿Defecto del generador, del clasificador o de las clases? Principalmente de las propias clases, con una contribución del generador:

- **Clases:** Shirt, Pullover y Coat son prendas de torso con manga larga muy parecidas (y T-shirt comparte la misma silueta de torso). En las imágenes, las filas generadas de Shirt y Pullover son casi indistinguibles (mangas largas, cuerpo oscuro con textura), y lo mismo pasa con Ankle boot y Sneaker (calzado de perfil similar). Las confusiones ocurren justo entre grupos visualmente parecidos.
- **Generador:** con $w = 1$ varias muestras salen ruidosas, de bajo contraste o con textura rara (se ve en la cuadrícula de la Task 3.3, p. ej. las filas de T-shirt y Dress), y esas muestras ambiguas son las que el clasificador confunde. La fidelidad sube a 0.952 con $w = 3$, es decir, parte del error es del condicionamiento débil de $w = 1$.
- **Clasificador:** su exactitud en validación es 0.8874, apenas sobre el 88 % pedido, así que también se equivoca. No calculé su matriz de confusión sobre datos reales; para confirmar que sus errores caen en los mismos pares habría que hacerlo.

(c) La guía corresponde a muestrear $\tilde{p}(x|c) \propto p(x)p(c|x)^w$

Demostración. Con $s(x_t \mid c) \approx -\epsilon_\theta(x_t, t, c)/\sqrt{1-\bar{\alpha}_t}$ y $s(x_t) \approx -\epsilon_\theta(x_t, t, \varnothing)/\sqrt{1-\bar{\alpha}_t}$, la predicción guiada $\tilde{\epsilon} = \epsilon_\varnothing + w(\epsilon_c - \epsilon_\varnothing)$ corresponde al score

$$\tilde{s} = s_\varnothing + w\,(s_c - s_\varnothing) = \nabla \log p(x) + w\,[\nabla \log p(x \mid c) - \nabla \log p(x)]\,.$$

Por la regla de Bayes, $p(x \mid c) = p(c \mid x)p(x)/p(c)$, y como $p(c)$ no depende de x, al tomar el gradiente del logaritmo:

$$\nabla \log p(x \mid c) - \nabla \log p(x) = \nabla \log p(c \mid x).$$

Entonces

$$\tilde{s} = \nabla \log p(x) + w\,\nabla \log p(c \mid x) = \nabla \log[p(x)\,p(c \mid x)^w]\,,$$

es decir, se muestrea de $\tilde{p}(x \mid c) \propto p(x)\,p(c \mid x)^w$. Para $w = 1$ es $p(x)p(c|x) \propto p(x|c)$ (el modelo condicional puro), y para $w = 0$ es $p(x)$ (incondicional).

Efecto sobre la diversidad. Elevar $p(c \mid x)$ a $w > 1$ amplifica las diferencias: las imágenes donde la clase es muy clara pesan mucho más y las ambiguas casi desaparecen (algo parecido a bajar la temperatura). La masa se concentra en pocos modos típicos de la clase y la diversidad baja. Es lo que se ve en los datos: diversidad $7.937 \to 7.880 \to 7.592$ para $w = 1, 3, 7$, con la caída más fuerte en Bag ($-50.8\,\%$). La fidelidad sube primero ($0.784 \to 0.952$), pero con $w = 7$ baja a 0.878: un factor tan grande lleva a pasos fuera de la distribución de datos y a imágenes saturadas.

(d) Intervalo de t con mayor y menor pérdida y relación con el SNR

MSE de validación por intervalo de t:

intervalo	1–100	101–200	201–300	301–400	401–500
MSE	0.2145	0.0826	0.0478	0.0287	0.0162
intervalo	501–600	601–700	701–800	801–900	901–1000
MSE	0.0080	0.0036	0.0017	0.0011	0.0010

La pérdida es **mayor en el intervalo 1–100 (0.2145)** y **menor en el 901–1000 (0.0010)**, unas 215 veces menor, y decrece de forma monótona. Los dos primeros intervalos concentran el 73 % de la suma de las pérdidas.

Relación con el SNR (Task 2.5b). Con SNR alto (t pequeño) $x_t \approx x_0$ y el ruido es una perturbación mínima que la red solo puede separar si reconoce el detalle fino de la imagen: la tarea es la más difícil. En el calendario lineal, los primeros $\sim$27 pasos tienen SNR > 100 y el SNR cruza 1 en $t = 260$. Con SNR bajo, $x_t \approx \epsilon$ y predecir ϵ se reduce a copiar x_t (reescalado): en el calendario lineal los 326 pasos con SNR < 0.01 ($t \geq 675$) cubren los intervalos 701–1000 y el final del 601–700, donde la pérdida es ≤ 0.0036.

¿Coincide con lo que predije? Sí. En 2.5(b) anticipé que los pasos de SNR muy alto serían los más difíciles y los de SNR muy bajo casi triviales, y la gráfica lo confirma. Además, estos últimos aportan muy poco gradiente cuando t se sortea uniformemente, lo que explica por qué el calendario propio (coseno), con 880 pasos en el rango informativo contra 647, es más eficiente.

Task 4 — Comparación de familias y pass@k

Task 4.1 — VAE vs. difusión

(a) Nitidez sobre 500 imágenes de cada fuente

	nitidez	% de la real
Real	0.0886	100 %
Difusión ($w = 1$)	0.0739	83 %
VAE ($\beta = 1$)	0.0508	57 %

(b) Tiempo de generación de 100 imágenes y evaluaciones de red

	tiempo (100 imgs)	evaluaciones de red
VAE ($\beta = 1$)	0.3 ms	1
Difusión ($w = 1$)	1785.6 ms	1000

Razón de tiempos difusión / VAE $\approx$ **6341**$\times$.

(c) Mapa de compensaciones y cómo acortar la difusión

- **VAE:** rápido (una pasada del decoder) y estable (pérdida determinista, sin adversario), pero borroso: con MSE el decoder predice $\mathbb{E}[x \mid z]$ y promedia los bordes (Task 1.4d).
- **Difusión:** más nítida (83 % vs. 57 % de la nitidez real) y también estable (un MSE sencillo), pero $\sim$6300$\times$ más lenta por sus $T = 1000$ evaluaciones secuenciales.

Ambas ganan en estabilidad; la compensación real es **nitidez vs. velocidad**.

Para acortar la difusión: muestrear con menos pasos (p. ej. DDIM con $\sim$50 pasos, $\sim$20$\times$ menos evaluaciones), destilar el modelo a pocos pasos (destilación progresiva o modelos de consistencia) o usar una U-Net más pequeña. Además, evitar $w \neq 1$, que duplica el batch y multiplica el costo por $\sim$2.4 (6.8 vs. 2.8 s / 100 imgs).

Task 4.2 — pass@k

(a) pass@1, pass@5 y pass@10 con el estimador insesgado

	pass@1	pass@5	pass@10
P1	0.0000	0.0000	0.0000
P2	0.1000	0.4474	0.7632
P3	0.2500	0.8063	0.9837
P4	0.4500	0.9702	0.9999
P5	0.7000	0.9996	1.0000
P6	1.0000	1.0000	1.0000
promedio	**0.4167**	**0.7039**	**0.7911**

P2, $k = 5$ ($n = 20$, $c = 2$, $n - c = 18 \geq 5$):

$$\text{pass@5} = 1 - \frac{\binom{18}{5}}{\binom{20}{5}} = 1 - \frac{8568}{15504} = 1 - 0.5526 = \mathbf{0.4474}.$$

(b) P2: pass@$k = 1 - \frac{(20-k)(19-k)}{380}$ **y menor** k **con pass@**$k > 0.9$

Demostración. Para $n = 20$, $c = 2$:

$$\frac{\binom{18}{k}}{\binom{20}{k}} = \frac{18!}{k!\,(18-k)!} \cdot \frac{k!\,(20-k)!}{20!} = \frac{(20-k)(19-k)}{20 \cdot 19} = \frac{(20-k)(19-k)}{380}$$

$$\Rightarrow \text{ pass@}k = 1 - \frac{(20-k)(19-k)}{380}.$$

La celda verifica que coincide con el estimador para $k = 1, \ldots, 20$.

Menor k: $k = 13$ da $1 - \frac{42}{380} = 0.8895$ y $k = 14$ da $1 - \frac{30}{380} = 0.9211$, así que $\mathbf{k = 14}$.

(c) Estimador insesgado vs. ingenuo, $k = 5$

	insesgado	ingenuo	diferencia
P1	0.0000	0.0000	0.0000
P2	0.4474	0.4095	0.0379
P3	0.8063	0.7627	0.0436
P4	0.9702	0.9497	0.0205
P5	0.9996	0.9976	0.0020
P6	1.0000	1.0000	0.0000

El ingenuo nunca supera al insesgado; la brecha es mayor en los problemas intermedios (P2, P3) y nula en los extremos (P1, P6).

Demostración. Si $n - c < k$, el insesgado vale $1 \geq$ ingenuo. Si $n - c \geq k$:

$$\frac{\binom{n-c}{k}}{\binom{n}{k}} = \prod_{i=0}^{k-1} \frac{n-c-i}{n-i} \leq \left(\frac{n-c}{n}\right)^k = \left(1 - \frac{c}{n}\right)^k,$$

y restando de 1 se invierte la desigualdad: ingenuo $\leq$ insesgado.

Intuición. El ingenuo muestrea con reemplazo; el insesgado, sin reemplazo, no repite soluciones fallidas y por eso es más probable que encuentre una correcta.

(d) Agente que reintenta hasta 10 veces con verificador perfecto

Con hasta 10 intentos y un verificador perfecto, el agente resuelve **P2–P6** (todos con $c > 0$), con probabilidad pass@10: 0.76 en P2, 0.98 en P3 y ≥ 0.9999 en P4–P6. **P1** ($c = 0$) no se resuelve nunca.

El promedio sube de **0.417** (pass@1) a **0.791** (pass@10), casi el doble: el modelo sabe resolver la mayoría de los problemas, pero no de forma consistente, y reintentar con verificación compensa esa inconsistencia. El techo es $5/6 \approx 0.833$ (fracción con $c > 0$); lo que falta (P1) requiere un mejor modelo, no más intentos.

Referencias

[1] Alexander Quinn Nichol y Prafulla Dhariwal. «Improved Denoising Diffusion Probabilistic Models». En: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. Vol. 139. PMLR. 2021, págs. 8162-8171. URL: https://arxiv.org/abs/2102.09672.